

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 02 · LECTURE NOTES

The Network Underneath

What the wire actually gives you, and what it charges.

MODULE OUTCOMES

- Compare TCP and UDP on the properties that matter.
- Explain head-of-line blocking and when it is fatal.
- Reason about bandwidth-delay product and congestion control.
- Explain NAT traversal and why peer-to-peer is hard.
- Choose a transport and justify it.

1 TCP and UDP

Property	TCP	UDP
Delivery	Reliable — lost segments are retransmitted until acknowledged.	Best effort. A lost datagram is simply gone; retrying is your decision and your code.
Ordering	Strictly in order — and that is the problem, per the callout below.	Any order, or none. You attach sequence numbers if you need them.
Framing	A byte stream with no message boundaries; you must frame it yourself, and forgetting to is a classic bug.	Datagrams, preserved whole. One send is one receive.
Congestion control	Built in and mandatory , and generally well tuned.	Entirely yours, and you must implement it. UDP traffic with no congestion control is antisocial and will be throttled.
Head-of-line blocking	Yes. One lost segment delays every byte behind it.	None. Each datagram is independent.
Connection	A handshake and per-connection state at both ends.	Stateless , which is why NAT and firewalls treat it badly (§3).

TCP'S ORDERING GUARANTEE IS WHAT MAKES IT WRONG FOR GAMES

TCP delivers bytes to the application in order. So if segment 7 is lost, segments 8, 9, and 10 may have *arrived* at the machine and be sitting in the kernel's receive buffer — but the application cannot have them until 7 is retransmitted and arrives.

That is one extra round trip of delay, minimum, on data you already physically have. At 150 ms RTT it is 150 ms of added latency applied to everything behind the loss.

For a file transfer this is exactly the right behaviour. You need every byte and you need them in order; there is no sense in which byte 8 is useful without byte 7.

For a real-time game it is exactly wrong. Position update 7 is *obsolete* the moment update 8 exists — nobody wants to know where the player was two frames ago. **Waiting for stale data before delivering fresh data is the worst available trade**, and it is precisely TCP's central promise. **The guarantee is not too weak; it is too strong**, which is an unusual and important shape of problem.

SO REAL GAMES SEND UDP AND REBUILD WHAT THEY NEED

Unreliable and unordered for state — positions, orientations, velocities. A dropped update is simply superseded by the next one, so retransmitting it would deliver something already worthless.

Reliable and ordered for events — a chat message, a weapon pickup, a death, a score change. These must arrive, and they must arrive once. **Implemented over UDP with sequence numbers, acknowledgements, and retransmission.**

So you end up rebuilding a subset of TCP, per channel, with the semantics each channel actually requires. The important word is *per channel*: a loss on the chat channel must not delay the position channel, which is precisely what TCP cannot offer.

That is what ENet, Valve's GameNetworkingSockets, and QUIC all do. QUIC is the mainstream acknowledgement that this analysis was right — HTTP/3 abandoned TCP in favour of independent streams over UDP for exactly the head-of-line-blocking reason, after the game industry had been doing it for twenty years.

2 Capacity

$$\text{BDP} = \text{bandwidth} \times \text{round-trip time}$$

The bandwidth-delay product is the number of bytes that must be 'in flight' — sent but not yet acknowledged — to keep a link fully utilised. **A 100 Mb/s link with a 100 ms RTT has a BDP of 1.25 MB**, so you must have that much unacknowledged data outstanding at all times or the link idles. **If the window is smaller than the BDP, throughput is capped at window / RTT** regardless of the link's capacity — which is why a fast connection to a distant server can be slow until window scaling takes effect, and why TCP slow start is so costly for short connections. It is also why QUIC's 0-RTT connection resumption matters.

BUFFERBLOAT: LATENCY CAUSED BY TOO MUCH BUFFER

Loss-based congestion control — Reno, CUBIC — increases its sending rate until packets are dropped. Loss is its only signal that the path is congested.

So it fills every buffer between the two endpoints before it backs off, and consumer routers and modems were built with very large buffers on the theory that dropping packets is bad.

Full buffers mean queueing delay. A saturated link can show round-trip times of one or two *seconds*, with no packet loss whatsoever — every packet arrives, having waited in a queue.

This is why a large download ruins a game on the same connection, and why BBR, CoDel, and fq_codel were developed — they treat delay, or queue occupancy, as the congestion signal rather than loss.

Adding buffer made latency dramatically worse, which is worth carrying as a general lesson: **a buffer trades latency for throughput, and that trade is not always the one you want** (CSCE 650 Module 06 made the same point about render queues).

3 NAT and peer-to-peer

Both peers are behind NAT. Neither has a public address. Neither can accept an inbound connection. There is no address for either one to dial.

STUN ask a public server "what address do I appear to come from?" -- learn your own mapped endpoint.

HOLE PUNCHING
both peers send to each other's mapped endpoint at the same time. Each OUTBOUND packet creates a NAT mapping that admits the other's INBOUND packet. Works for cone NATs.

TURN relay everything through a public server. ALWAYS works. Costs bandwidth and adds a latency hop. The fallback, not the plan.

ICE try all of the above in parallel and use whatever works. WebRTC does this; so should you.

SYMMETRIC NAT uses a DIFFERENT external port per destination, so the address STUN taught you is wrong for any other peer. Hole punching fails. Roughly 8-20% of users. You need TURN.

Symmetric NAT is why every peer-to-peer system needs relay servers anyway, which substantially undercuts the usual argument that P2P saves on server costs. **You still run infrastructure, you still pay for bandwidth, and you now have two code paths to test instead of one** — and the relayed path is the one your least-well-connected users will be on. The honest framing is that P2P reduces bandwidth cost for the majority and adds complexity for everyone.

4 Choosing a transport

If you need	Use	Because
Real-time state updates	UDP, unreliable, unordered.	Stale data is worthless and head-of-line blocking is fatal (§1).
Real-time state <i>and</i> reliable events	UDP with a per-channel reliability layer — ENet, GameNetworkingSockets, or QUIC.	Do not write your own. These libraries already got the congestion control right, and congestion control is subtle enough that getting it wrong makes you a bad network citizen.
RPC between internal services	gRPC over HTTP/2.	Streaming, deadlines, cancellation, and mature tooling. Worth the dependency.
Anything web-facing	HTTP/3, which is QUIC.	Solves head-of-line blocking per stream , and 0-RTT resumption removes a round trip from reconnections.

If you need	Use	Because
Bulk transfer	TCP.	Ordering is exactly what you want, and the congestion control is well tuned for it.
Real-time from a browser	WebRTC data channels.	The only way to get UDP semantics in a browser , and it brings ICE (§3) with it.

MEASURE ON A BAD NETWORK, OR YOU HAVE NOT MEASURED

Localhost has about 0.05 ms of latency, no loss, no jitter, no reordering, and effectively infinite bandwidth. It is not a network; it is a memory copy with a socket API.

Test at 50, 150, and 300 ms RTT, with 1%, 5%, and 10% loss, and with jitter — because mobile networks routinely deliver all of these simultaneously, and your users are on them.

And test asymmetry and reordering. Consumer uplinks are a fraction of downlinks, and packets genuinely arrive out of order on multipath routes. **Both break naive implementations** in ways that are hard to diagnose after the fact.

This is the same rule as CSCE 735's insistence on measuring the real workload and CSCE 620's observation that random input proves nothing. The clean case is not the informative one — three courses, three domains, one conclusion, which suggests it is a fact about testing rather than about networks.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Glenn Fiedler — UDP vs TCP, and Reliability over UDP (free)	The §1 argument , made concretely and with code. The whole series is worth reading.
Ilya Grigorik — High Performance Browser Networking (free book)	Free in full. The best treatment of §2 — BDP, congestion control, and why latency dominates — and strong on WebRTC for §3.
Gettys & Nichols — Bufferbloat: Dark Buffers in the Internet (free)	The §2 callout, from the people who identified and named the problem.
RFC 8445 (ICE), and Valve's GameNetworkingSockets (free)	The §3 traversal machinery, and a production implementation of the §4 recommendation with readable source.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Demonstrate head-of-line blocking:** send numbered messages over TCP with 5% loss and plot delivery time against sequence number.
- 2 Send the same over UDP and compare the plots.
- 3 Implement sequence numbers and acknowledgements over UDP for a reliable channel.
- 4 **Add a second, unreliable channel** and show that loss on one does not delay the other.
- 5 **Compute the BDP** for three real links you have access to, and measure whether you achieve full throughput.
- 6 **Reproduce bufferbloat:** saturate your uplink and measure RTT during the transfer. Report the before and after.
- 7 Enable fq_codel if you can and measure again.
- 8 Implement STUN and determine your own NAT type.
- 9 **Attempt hole punching** between two machines on different networks and report whether it worked.
- 10 Measure a protocol of yours at 50, 150, and 300 ms RTT with jitter, and plot the degradation.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare TCP and UDP on six properties.
- 2 Explain head-of-line blocking and why TCP's guarantee is too strong for games.
- 3 What two channel types does a game need, and why?
- 4 Define bandwidth-delay product and say what happens when the window is smaller.
- 5 Explain bufferbloat and why more buffer made latency worse.
- 6 Describe STUN, hole punching, TURN, and ICE.
- 7 Why does symmetric NAT undercut the P2P cost argument?
- 8 Give six transport choices with reasons.